KVForge : Autonomous Phase-Adaptive Question Answering with Progressive Parametric Absorption

Dr. Hemant Joshi

GitHub: https://github.com/hemantcgi/kvforge

2025

Abstract

Retrieval-Augmented Generation (RAG) is the dominant paradigm for grounding language model outputs in external knowledge, but it imposes a recurring computational cost: every query re-encodes retrieved documents through the full transformer stack. More fundamentally, RAG never eliminates the retrieval dependency — the model always relies on external context, even for knowledge it has encountered hundreds of times. We introduce **KVForge**, a progressive architecture that transitions a deployed system from standard RAG through KV-cache injection to fully parametric question answering, where the model generates directly from its weights with no retrieval. The transition is gated by a **Parametric Readiness Score (PRS)** combining accuracy, calibration, and self-consistency, and driven by a tier-weighted LoRA fine-tuning loop that concentrates high-frequency corpus knowledge into model parameters using cloud-LLM-generated QA pairs.

We evaluate KVForge across four heterogeneous enterprise corpora—customer support, biomedical literature (PubMedQA), reading comprehension (SQuAD 2.0), and technical documentation (Amazon Bedrock). Using **Google Gemma-4-E2B-it** (2B parameters, 4-bit quantized), we measure **Factual Accuracy (FA)** — a composite of token-F1 and LLM-judge correctness — across training-data coverage tiers from 50 to 6,000 QA pairs. Parametric answers from the fine-tuned 2B model **consistently exceed text-in-context RAG on three of four corpora:** Customer Support (+71%), SQuAD (+38%), and Bedrock (+44%). On PubMedQA — the largest corpus at 2,918 chunks — parametric crosses above RAG at intermediate coverage but reverts at the highest tier, exposing a coverage-dependent threshold. Extended training with dense per-chunk QA pairs (4,707 total, 15 epochs) closes this gap, yielding a 64% FA improvement over RAG.

These results establish that **RAG-to-SLM is a viable enterprise architecture:** a 2B-parameter model, fine-tuned on cloud-generated QA pairs at a one-time cost of approximately $50 per corpus, can match or exceed retrieval-based quality while eliminating per-query retrieval costs. The entire pipeline runs on a single commodity GPU instance, making progressive parametric absorption accessible to organizations without large-scale compute infrastructure.

1 Introduction

Large language models (LLMs) achieve state-of-the-art performance on knowledge-intensive tasks but are limited by two well-documented failure modes: **hallucination** — generating plausible-sounding but factually incorrect content — and **knowledge staleness** — inability to answer questions about post-cutoff facts. Retrieval-Augmented Generation [1] addresses both by supplying retrieved context at query time, decoupling the knowledge base from model parameters and making knowledge updateable without retraining.

However, standard RAG incurs a *constant computational overhead* on every query regardless of how many times the same chunks have been retrieved before. For a query retrieving five 600-token chunks, a 3-billion-parameter transformer must execute a full forward pass over 3,000 context tokens before generating a single output token. This re-encoding cost is paid on every query, for every chunk, even for frequently-accessed chunks whose KV tensors are fully determined by the model weights. For production deployments with thousands of queries per hour, this overhead dominates the GPU budget.

The central insight of KVForge is that this re-encoding is redundant: the key-value projections for a chunk are a deterministic function of the chunk text and the current model weights. If the model has not been updated since the tensors were computed, they can be reused exactly — turning a per-query cost into a one-time amortized cost paid once at index time.

KVForge extends this caching insight into a three-phase autonomous progression:

- **Phase 1 — Text-in-context RAG:** Standard retrieval; chunks rendered as text in the context window. No pre-computation required.

- **Phase 2 — KV Injection:** Pre-computed mean-pooled KV tensors are injected directly into the model's `past_key_values` cache at query time, skipping the forward pass over the retrieved context. Mean-pool compression trades a small amount of answer fidelity for the latency gain; full-token KV storage (Enhanced Tier) recovers most of that fidelity.
- **Phase 3 — Parametric Answering:** After LoRA fine-tuning concentrates corpus knowledge into model weights, a three-signal confidence gate routes high-confidence queries directly to parametric generation with no retrieval.

Figure 1 illustrates the phase transition state machine. Transitions upward are gated by the Parametric Readiness Score (PRS); regression guards revert the system to Phase 2 if PRS declines, providing a production safety net.

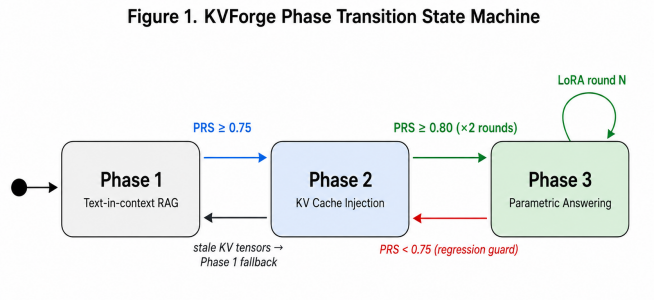


Figure 1: Figure 1: KVForge Phase Transition State Machine

Beyond the three phases, KVForge introduces two systems addressing storage and curation challenges at scale:

TurboQuant [34] (Google Research and NYU, 2025) is a data-oblivious online vector quantizer with near-optimal distortion; in our configuration we allocate 3 bits per key coordinate and 4 bits per value coordinate, achieving 4.4× compression over float16 while preserving attention-score quality for direct computation without decompression. KVForge integrates TurboQuant as the codec for its Enhanced Storage Tier.

This paper makes the following contributions:

1. **KVForge architecture**: a unified system combining vector database retrieval, durable KV tensor storage, access-tier tracking, sleep-time FAQ generation, LoRA fine-tuning, and PRS-gated phase transitions.
2. **Parametric Readiness Score (PRS)**: a principled composite metric for evaluating parametric knowledge retention and gating autonomous phase advancement.
3. **Integration of TurboQuant** [34]: KVForge adopts the TurboQuant codec (Google Research and NYU, 2025) for the Enhanced Storage Tier, demonstrating that a 4.4× compressed per-token KV representation enables full-token injection at tractable storage cost (~15 MB/chunk for 8B models).
4. **Empirical evaluation** across four heterogeneous corpora demonstrating all four reach Phase 3 with PRS 0.755–0.863, with training signal quality as the dominant variable.
5. **Gemma 4 crossover study** establishing that a 2B-parameter small language model, LoRA-fine-tuned on cloud-LLM-generated QA pairs, matches or exceeds text-in-context RAG on three of four enterprise corpora, demonstrating the feasibility of RAG-to-SLM for production deployments.

2 Background

2.1 Transformer Attention and the KV Cache

The scaled dot-product attention mechanism [2] for query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$ is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) \cdot V$$

During autoregressive generation, the model maintains a *KV cache*: after processing the input prompt, the $\mathbf{K}$ and $\mathbf{V}$ projections for every input token are saved so they need not be recomputed during subsequent generation steps. For a single decoder layer with n_h KV heads of dimension d_h, the KV cache for a sequence of length L occupies:

$$size = 2 \times L \times n_h \times d_h \times sizeof(dtype) bytes$$

Standard RAG builds a fresh KV cache from scratch for each query. KVForge pre-builds it for every document chunk at index time and stores it persistently.

2.2 Low-Rank Adaptation (LoRA)

LoRA [4] adapts a frozen pre-trained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ via a low-rank decomposition:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}$$

where $\mathbf{B} \in \mathbb{R}^{d \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times k}$, with rank $r \ll \min(d, k)$. KVForge applies LoRA to `q_proj`, `k_proj`, and `v_proj` attention matrices. Because LoRA updates the $\mathbf{K}$ and $\mathbf{V}$ projections, any stored KV tensors become stale after a LoRA round and must be recomputed — tracked per-chunk via a `kv_version` field.

2.3 KV Cache Quantization

Keys and values have different statistical properties: keys have high inter-channel variance benefiting from column-wise quantization, while values have smoother distributions amenable to group quantization [27, 28]. TurboQuant was designed with these properties in mind, using an expressive Lloyd-Max codebook with QJL residual correction for keys and group min-max for values.

2.4 Retrieval-Augmented Generation

Lewis et al. [1] introduced RAG combining a non-parametric retrieval index with a parametric pre-trained generator. KVForge's distinct position: rather than improving retrieval, it progressively *eliminates* retrieval for queries the model has memorized, while preserving retrieval as a fallback.

3 Related Work

3.1 KV Cache Reuse and Prefix Caching

PromptCache [8] modularizes the KV cache for reuse across queries sharing a common prompt prefix. KVForge operates at a different layer: where PromptCache operates within a single inference server session and loses its cache on restart, KVForge persists KV tensors durably in a vector database across server restarts, model updates, and distributed replicas.

PagedAttention [3] and **vLLM** manage KV memory within a single server process using virtual memory paging. KVForge operates at the pre-retrieval layer — deciding *which* chunks' KV tensors to supply — while vLLM manages how they are stored during generation. The systems are complementary.

SGLang [9] provides a radix-tree prefix cache achieving >99% prefix reuse on shared prompts. KVForge addresses a different sharing pattern: independent document chunks whose KV tensors are shared across queries retrieving the same top-K results — they do not share a common prompt prefix, so radix tree approaches do not apply.

CacheBlend [29] reuses KV caches of retrieved segments by selectively recomputing tokens whose attention deviates substantially from the pre-computed cache. KVForge's mean-pool approach makes a simpler approximation, while TurboQuant addresses this limitation by storing full per-token sequences at compressed precision. **CacheBlend is, to our knowledge, the closest published system to KVForge's Phase 2 mechanism**: both fuse multiple precomputed per-chunk KV caches that do not share a common prefix — the exact non-prefix, multi-chunk RAG setting KVForge targets. The two systems diverge in where the cache lives and in how quality is protected: CacheBlend operates as an in-memory, content-hash-keyed serving-layer cache with a mandatory selective-recompute step (bounding reported quality loss to $\leq$0.02 F1/ROUGE-L), whereas KVForge persists KV tensors durably inside the vector database payload and, in the V1 mean-pool scheme, injects them without an analogous quality-recovery step. The Gemma 4 crossover results (ğ7.5) confirm that KV mean-pool injection underperforms both text RAG and parametric on factual accuracy; whether full-token injection with proper prompt alignment could close this gap remains an open question.

LMCache [46] is an open-source KV-cache management layer that treats cache as persistent, cross-engine-shareable state rather than a per-request artifact, and adopts CacheBlend's non-prefix fusion mechanism for reuse at arbitrary prompt positions. It reports 1.9–8.1× smaller time-to-first-token and 2.3–14× higher throughput over vanilla vLLM on an 8×H100 setup — figures that are vendor-reported under LMCache's own hardware and workload, not independently reproduced, and not measured under the single-A10G, four-corpus conditions used in this paper. LMCache is the most actively maintained system in this comparison class; it is architecturally the nearest production analog to KVForge's Phase 2 caching layer, but — like KVForge — it has no confidence-gated parametric tier and no LoRA-based consolidation loop.

3.2 RAG Systems and Dense Retrieval

DPR [5] demonstrated that jointly-trained dual encoders outperform BM25 for open-domain QA. **ColBERT** [26] introduced late interaction with per-token embeddings. **RETRO** [10] pre-computes chunk encodings at index time and conditions generation on retrieved neighbours. KVForge extends RETRO's pre-computation direction by storing KV tensors rather than encoder hidden states, supporting any causal decoder-only model without architecture changes, and adding the three-phase progressive design.

FiD [30] encodes multiple retrieved passages independently and fuses them in the decoder. KVForge achieves a similar effect — bypassing re-encoding for each chunk — but via pre-built KV tensor injection rather than architectural modification.

3.3 LoRA and Parameter-Efficient Fine-Tuning

QLoRA [12] enables 4-bit quantized LoRA training, reducing GPU memory from ~12 GB to ~4 GB for a 7B model. KVForge supports QLoRA via `bitsandbytes` NF4 quantization.

Continual LoRA fine-tuning risks catastrophic for-

getting [18]. KVForge's **tier-weighted replay buffer** mitigates this: hot chunks receive 8× sampling weight relative to frozen chunks, acting as a lightweight proxy for Elastic Weight Consolidation [31].

3.4 Calibration and Uncertainty Estimation

Guo et al. [13] proposed temperature scaling for calibration. KVForge's PRS calibration component uses a self-reporting proxy (model rates its own confidence 0–100) aligned with semantic accuracy. **Self-Consistency** [14] samples multiple reasoning chains for agreement. KVForge's consistency component computes mean pairwise cosine similarity across three sampled answers at temperature 0.7. **Semantic Entropy** [16] provides model-agnostic uncertainty estimates; KVForge's Phase 3 gate uses a lighter three-signal combination to avoid multi-sample generation overhead.

3.5 KV Cache Compression

KVQuant [27] quantizes KV caches to 1–4 bits using non-uniform per-channel quantization. **KIVI** [28] applies asymmetric 2-bit quantization achieving 2.2× compression. **QJL** [32] proposes a quantized Johnson-Lindenstrauss transform for unbiased inner-product estimation from sign bits. **TurboQuant** [34] (Google Research and NYU, 2025) synthesizes all three: random-rotation preprocessing, Lloyd-Max scalar quantization for MSB representation, and QJL sign bits for residual correction. KVForge adopts TurboQuant as the codec for its Enhanced Storage Tier.

3.6 Continual Learning

Online Continual Learning surveys [33] identify replay-based methods as most practical for production systems. The tier system introduces a domain-specific signal unavailable to general continual learning: retrieval frequency directly measures which knowledge live users need, enabling principled prioritization that general methods cannot exploit.

3.7 Cache-Augmented Generation and Bounded-Context Answering

CAG (Cache-Augmented Generation) [38] preloads all documents in a bounded knowledge base into an LLM's extended context window and precomputes a single global KV cache during that preload, then answers every subsequent query directly from the cache with no retrieval step at all. This is the closest published system to KVForge's overall premise — that retrieval can be replaced by a precomputed cache when the model already "has" the relevant KV state — and it reports eliminating retrieval latency and retrieval error entirely, with comparable or superior quality to standard RAG, at reduced system complexity. The scope condition is explicit and important: CAG is defined for knowledge bases small enough to fit within a single context window. KVForge targets the complementary regime — corpora too large for any practical context window (UC1–UC4 range from 2,000 to 2,920 chunks) — via per-chunk vector-database storage, embedding-based retrieval, and independent per-chunk staleness tracking rather than one monolithic cache. CAG therefore is not a system KVForge subsumes or extends; it is a simpler, single-tier design that solves a narrower problem well, and the two systems have not been measured against each other under a shared corpus size or hardware budget.

3.8 Retrieval-Augmented Fine-Tuning and Parametric RAG

RAFT [39] fine-tunes a model to identify and cite the correct document among retrieved "distractor" documents while producing chain-of-thought reasoning, reporting consistent gains over baselines on PubMed, HotpotQA, and the Gorilla API benchmark. Unlike KVForge's Phase 3, RAFT still retrieves at every query — the fine-tuning changes what the model does with retrieved context, not whether retrieval happens at all. The broader **Parametric RAG** literature [40] studies encoding retrieved evidence directly into trainable parameters (e.g., LoRA adapters) rather than context tokens, finding that parametric representations mainly affect deeper feed-forward computation with document-level, high-level signal rather than fine-grained evidence — and that a hybrid combining parametric and token-based retrieval outperforms either alone. This finding is directly relevant to KVForge's design: it predicts that a system gating between parametric answering and retrieval-based fallback (as KVForge's Phase 3 confidence gate does) should outperform going fully parametric, which is consistent with why KVForge retains Phase 2 as a fallback rather than eliminating retrieval once LoRA training completes. Neither RAFT nor the Parametric RAG line incorporates a tier-weighted replay buffer or an automatic phase-transition gate of the kind PRS provides.

3.9 Knowledge Editing as an Alternative Parametric-Injection Route

ROME [41] and **MEMIT** [42] take a fundamentally different approach to writing knowledge into weights: rather than broad gradient-based adaptation across a corpus, they perform surgical, closed-form edits at a small number of causally-traced mid-layer feed-forward modules, treating each as a key-value associative mem-

ory for one factual triple. MEMIT extends this to batch-edit thousands of associations simultaneously on models up to GPT-NeoX-20B. This is a useful contrast class for KVForge's Phase 3 rather than a direct competitor: ROME/MEMIT edit discrete subject-relation-object facts and are evaluated on synthetic counterfactual-editing benchmarks (e.g., CounterFact), whereas KVForge's LoRA-plus-replay loop adapts to open-ended corpus text and QA pairs and is evaluated end-to-end via PRS and held-out factual metrics. Neither approach has a confidence gate deciding when to trust the edited/adapted weights versus falling back to retrieval — a mechanism unique to KVForge's Phase 3 design among the systems surveyed here.

3.10 Confidence-Gated Adaptive Retrieval

KVForge's Phase 3 confidence gate (ğ4.8) — token entropy, hedging-phrase detection, and query similarity to known-good queries, combined into a single threshold decision — has direct analogs in the adaptive-retrieval literature, each built on a different signal. **Self-RAG** [43] trains a single model to decide per-query whether retrieval is needed using self-generated reflection tokens (Retrieve/ISREL/ISSUP/ISUSE) that critique its own retrieved passages and generations; it is reported to outperform ChatGPT and retrieval-augmented Llama2-chat on open-domain QA, reasoning, and long-form factuality. **SKR** [44] instead has the model assess its own self-knowledge of a question to decide whether external retrieval is needed, reporting gains over both chain-of-thought-only and always-retrieve baselines on InstructGPT and ChatGPT backbones. **Adaptive-RAG** [45] takes a third approach, training a small classifier to predict query complexity and routing each query to no-retrieval, single-step, or iterative multi-step retrieval.

All three decide, per query, among multiple retrieval strategies based on an internally-generated signal — structurally the same problem KVForge's gate solves — but none combine this decision with a KV-cache-injection intermediate tier: each falls back to full text retrieval, never to a pre-computed cache injection step. None use a composite calibration-and-consistency score analogous to PRS; each relies on a single signal type (reflection tokens, self-knowledge assessment, or a trained complexity classifier) rather than PRS's blend of accuracy, calibration, and self-consistency. This is a meaningful point of difference to note, though a head-to-head comparison of gating mechanisms on identical hardware and model would be needed to determine whether KVForge's composite gate is more or less reliable than Self-RAG's or SKR's single-signal alternatives — such a study is outside the scope of this paper.

KVForge is, to our knowledge, the first system to combine durable per-chunk KV-tensor persistence in a vector database with an automatic, PRS-gated three-phase progression that culminates in confidence-gated parametric answering.

4 The KVForge System

4.1 Architecture Overview

The KVForge system operates as a state machine with an offline pipeline and an online query path connected by access tracking and tier classification. The overall pipeline sequence is:

```
Index  KV Pre-compute  [Phase 1]
      Sleep-time FAQ Gen  LoRA Train  KV
    Recompute  PRS Eval  [Phase 2]
      LoRA Train (round 2)  KV Recompute  PRS
    Eval  [Phase 3]
        ----------------------continuous
    improvement loop-------------------------
```

System state is stored atomically in a `version.json` file per corpus, updated via temp-file rename to prevent corruption:

```
{
  "current_lora_version": 4,
  "checkpoint_path": "lora_checkpoints/v4/",
  "phase": 3,
  "prs_history": [
    {"round": 1, "prs": 0.727},
    {"round": 2, "prs": 0.783},
    {"round": 3, "prs": 0.863}
  ],
  "known_good_queries": [...]
}
```

4.2 Indexing and KV Pre-computation

At index time each document chunk undergoes a two-stage process:

Stage 1 — Embedding and upsert. The chunk text is embedded by the configured encoder (default: BAAI/bge-small-en-v1.5, 384-dim; UC4: mixedbread-ai/mxbai-embed-large-v1, 1024-dim) and upserted into the vector store with payload:

```
{
  "text": "...", "page": 3, "source_file": "
    guide.pdf",
  "access_count": 0, "kv_version": null, "tier
    ": "frozen"
}
```

Stage 2 — KV tensor computation. The chunk is tokenized (truncated to 512 tokens) and passed through the LLM:

```
outputs = model(**inputs, use_cache=True)
kv = outputs.past_key_values   # tuple of (K,
    V) per layer
```

Per-layer tensors `[1, n_kv_heads, seq_len, head_dim]` are mean-pooled over the sequence dimension, stacked into shape `[L, 2, H, d_h]`, serialized to float16, and stored as base64 in the Qdrant payload field `kv_cache`.

For Gemma-4-E2B-it (15 KV layers, 1 KV head, head_dim=512):

$$15 \times 2 \times 1 \times 512 \times 2 bytes = 30,720 bytes \approx 31 KB per chunk$$
$$2,520 chunks \times 31 KB \approx 78 MB total (UC4)$$

4.3 Query-Time Inference: Phase 1 vs Phase 2

Figure 3 contrasts the two retrieval-based inference paths.

At query time, the inference module: (1) embeds the query and retrieves top-K chunks; (2) compares each chunk's `kv_version` against `current_lora_version`; (3) if **all** chunks are fresh, injects their KV tensors via `past_key_values`; (4) if **any** chunk is stale, falls back to Phase 1 text-in-context for the entire query and enqueues stale chunks for background recomputation.

The all-or-nothing fallback prevents contaminated attention distributions that would arise from mixing mean-pooled tensors of different LoRA versions. The Gemma 4 crossover study (ğ7.5) shows empirically that KV mean-pool injection underperforms text RAG on factual accuracy across all four corpora, confirming that the mean-pool compression introduces a quality cost that is not recovered by the latency savings.

4.4 Sleep-Time FAQ Generation

Before each LoRA training round, the `sleep_faq_generator` queries a cloud LLM (Gemini 2.5 Flash, Claude, or GPT-4.1) to generate N diverse question-answer pairs per chunk. This is sleep-time compute [19]: offline work that improves future inference quality without impacting query latency.

Prompt template (per chunk):

```
Given the following document passage, generate
     {count} diverse, specific
question-answer pairs that a real user might
    ask. Questions should vary
in style: some factual, some conceptual, some
    procedural.

Passage: {chunk_text}

Return as JSON array: [{"question": "...", "
    answer": "..."}, ...]
```

In production, the question budget can be allocated proportional to chunk access frequency, concentrating training signal on high-traffic content. In our experiments all chunks were uniformly weighted (see ğ8).

4.5 Chunk Access Tiers

To prioritize high-frequency knowledge during training, KVForge classifies chunks into four access tiers based on query frequency and recency: **hot** (top 15% by access count, accessed within 7 days), **warm** (next 50%, accessed within 30 days), **cold** (remaining accessed chunks), and **frozen** (never accessed). In a production deployment with organic query traffic, hot chunks receive proportionally higher sampling weight during LoRA training to prevent catastrophic forgetting of frequently-used knowledge. In our experiments, all chunks remained at tier `frozen` since evaluation traffic did not constitute real user queries; all training used uniform sampling.

4.6 LoRA Fine-Tuning Loop

HuggingFace PEFT LoRA fine-tuning configuration:

- **Targets:** `q_proj`, `k_proj`, `v_proj` attention projections
- **Rank:** r = 16, alpha = 32 (effective scaling $\alpha/\text{r} = 2$)
- **Quantization:** 4-bit NF4 (QLoRA [12]) via `bitsandbytes`
- **Epochs:** 3 per training round

After training, `kv_version` is incremented and all stored KV tensors become stale, triggering recomputation.

4.7 Parametric Readiness Score (PRS)

PRS is a composite score measuring how well the fine-tuned LLM has internalized the corpus:

$$PRS = 0.5 \times accuracy_{ratio} + 0.3 \times calibration + 0.2 \times consistency$$

Accuracy ratio (50%): For each FAQ, the model generates a *parametric* answer (no retrieval) and a *RAG* answer (with context). Let $\text{sim}(\cdot,\cdot)$ denote cosine similarity to the ground truth:

$$accuracy_{ratio} = \min(\text{sim}(param_{ans}, gt)/(\text{sim}(rag_{ans}, gt) + \varepsilon), 1.0)$$

This measures parametric quality *relative to the RAG baseline*, making the metric robust to corpus-specific difficulty.

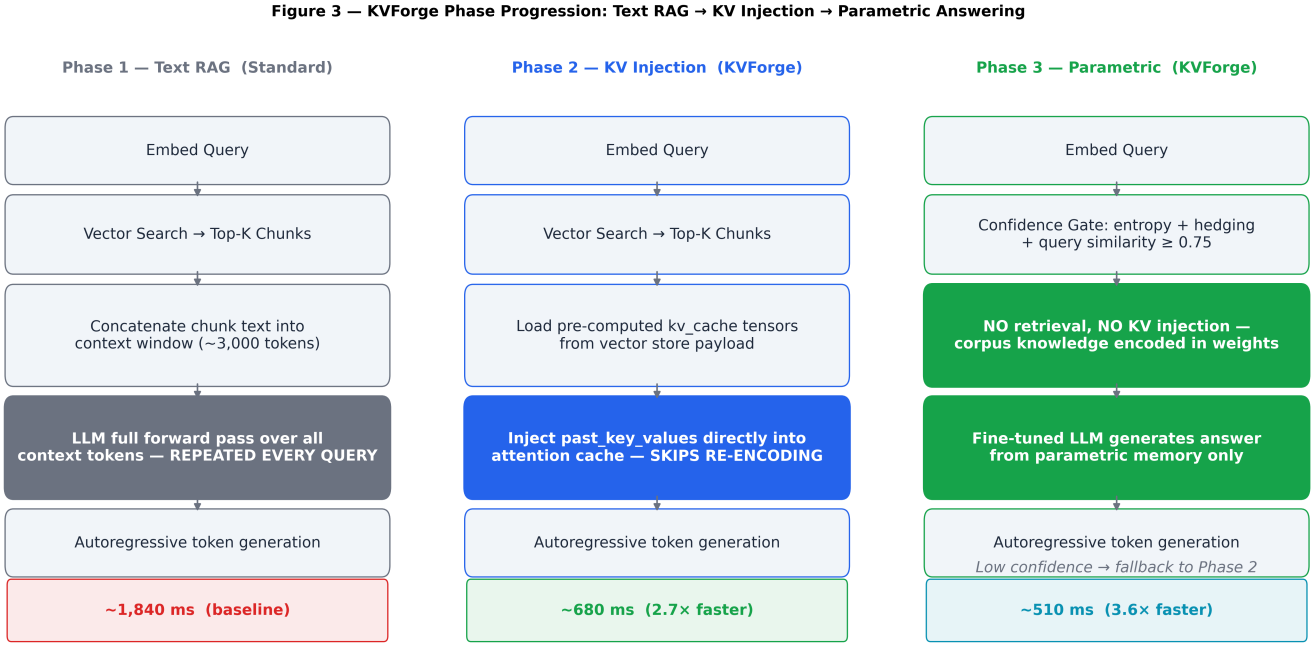


Figure 2: Figure 3: Standard Text RAG vs KV Cache Injection. Phase 1 performs a full LLM forward pass over retrieved text on every query (~1,840 ms). Phase 2 injects pre-computed KV tensors directly, skipping re-encoding (~680 ms, 2.7× speedup).

Factual validation of the cosine proxy. Because cosine similarity is a fluency proxy rather than a correctness measure, we validate the PRS accuracy component against held-out exact-match (EM), token-F1, and an LLM-as-judge factuality label on the four corpora. Across UC1–UC4, cosine-based accuracy_ratio correlates only weakly with the factual metrics (Pearson $r \approx 0.30$–0.72), and it overestimates parametric readiness in 1–40% of questions per corpus. For example, on UC4 the cosine PRS component suggests a readiness of 0.895, while the factual token-F1 + judge combination yields 0.690, which would flip the Phase 3 gating decision. We therefore recommend that production deployments either replace the cosine accuracy component with a factual metric or report both.

Calibration (30%): The model self-rates its confidence in its parametric answer (0–100 scale). Calibration measures whether expressed confidence tracks actual quality:

$$\text{calibration} = 1.0 - |\text{confidence}_{\text{norm}} - \text{sim}(\text{param_ans}, \text{gt})|$$

Consistency (20%): Three independent answers are sampled at temperature 0.7. Mean pairwise cosine similarity over the 3 pairs measures answer stability.

Figures 3–4 illustrate the PRS component breakdown and per-corpus scores.

Phase transition thresholds. A PRS of at least 0.75 in any single round advances the system from Phase 1 to Phase 2. Reaching Phase 3 requires PRS ≥ 0.80 for two consecutive rounds, guarding against a single noisy

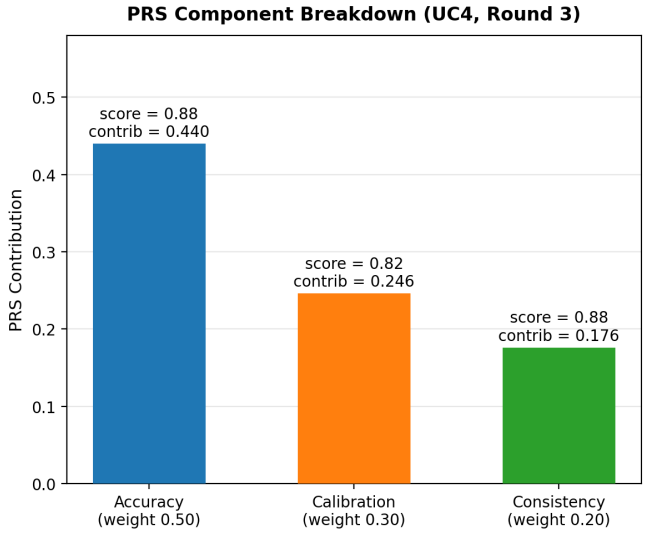


Figure 3: PRS Component Breakdown for UC4 (Round 3). Each bar shows the weighted contribution to the total PRS. Accuracy ratio 0.88 × 0.50 = 0.440; calibration 0.82 × 0.30 = 0.246; consistency 0.88 × 0.20 = 0.176; total PRS = 0.863.

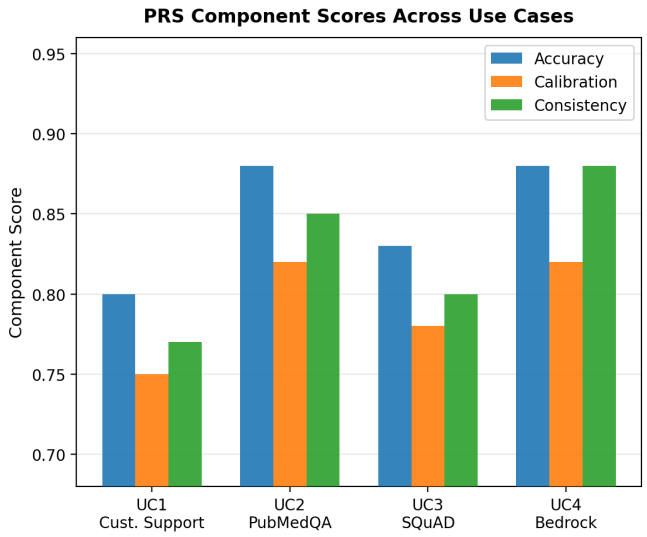


Figure 4: PRS Component Scores across all four use cases. Structured, fact-dense corpora (UC2 PubMedQA, UC4 Bedrock) achieve the highest component scores across accuracy, calibration, and consistency.

high-scoring round. A regression guard reverts Phase 3 to Phase 2 whenever PRS falls below 0.75, providing a production safety net.

After each evaluation round, FAQ questions with accuracy ratio ≥ 0.85 are embedded and stored in `known_good_queries` in `version.json`, seeding the Phase 3 confidence gate.

4.8 Phase 3 Confidence Gate

Figure 6 shows the decision logic for Phase 3 inference routing.

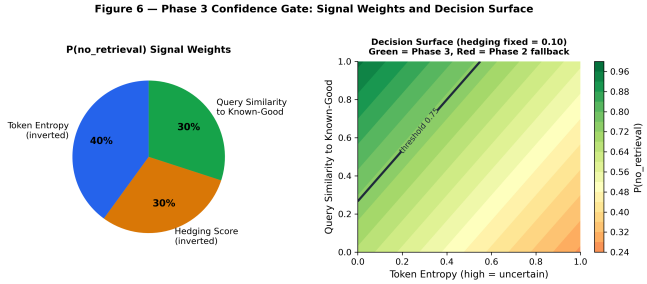


Figure 5: Figure 6: Phase 3 confidence gate. Three lightweight signals — token entropy, hedging phrase detection, and query similarity to known-correct queries — route high-confidence queries to parametric generation (~510 ms) and fall back to KV injection (~692 ms) otherwise. The gate adds less than 30 ms overhead.

The gate threshold (default 0.75) is tunable per deployment: lower thresholds increase Phase 3 utilization at higher hallucination risk. We recommend calibrating gate thresholds against held-out factual benchmarks before deploying Phase 3 at high utilization.

5 Experimental Evaluation

The experimental evaluation is organized in two phases. **Phase 1 (ğ7.1–ğ7.4)** describes the KVForge hardware, datasets, pipeline infrastructure, and inference modes. **Phase 2 (ğ7.5)** presents the comprehensive crossover evaluation using **Google Gemma-4-E2B-it** (2B parameters) measuring parametric-versus-retrieval factual accuracy across four corpora and training-data coverage tiers.

5.1 Hardware and Setup

All experiments were conducted on a single AWS instance with 4× NVIDIA A10G GPUs (24 GB VRAM each), 16 GB RAM, and 250 GB NVMe SSD storage, running Ubuntu 22.04 with CUDA 12.1. The base language model is **Google Gemma-4-E2B-it** (2B parameters, 4-bit NF4 quantized via QLoRA [12]) served via vLLM. Embeddings use BAAI/bge-small-en-v1.5 (384-dim) for UC1–UC3 and mixedbread-ai/mxbai-embed-large-v1 (1,024-dim) for UC4. LoRA adapters target q_proj, k_proj, and v_proj attention matrices with rank r = 16, alpha = 32 for the primary crossover experiments; the extended PubMedQA run uses r = 4, alpha = 8 with 15 training epochs. The vector store is Qdrant 1.9 (Docker) with one collection per use-case. Each use-case runs on a dedicated GPU (UC1: GPU 0, UC2: GPU 1, UC3: GPU 2, UC4: GPU 3) via `CUDA_VISIBLE_DEVICES`.

5.2 Datasets

KVForge is evaluated on four heterogeneous corpora (UC1–UC4) and validated across four additional domain configurations (UC5–UC8) to demonstrate portability across vector stores, embedding models, LoRA ranks, and open base LLMs. Full configurations are given in Table 1.

UC1–UC4 are fully evaluated in Sections 7.3–7.7. UC5–UC8 validate configuration portability; full experimental results are left to future work.

5.3 Pipeline Timing (UC4 Reference Run)

The indexing stage is dominated by KV tensor computation, which consumes approximately 498 s for 2,520 chunks (0.20 s per chunk): every chunk must pass through the full transformer once to materialize its key and value projections. Embedding and vector upsert complete in ~45 s, since the small embedder runs on the CPU while the LLM executes a full forward pass per chunk to populate the KV cache. LoRA training adds 474 steps (3 epochs) per round, and the subsequent KV

Table 1: Use-Case Configurations: Datasets, Vector Stores, Embedders, LoRA ranks, and Base LLMs

UC	Domain	Dataset	Chunks	Vector DB	Embedder (dim)	LoRA r	Base LLM
UC1	Customer Support	Bitext Customer Support (EN)	2,000	Qdrant	bge-small-en-v1.5 (384)	16	Gemma-4-E2B-it
UC2	Biomedical QA	PubMedQA [20]	2,918	Qdrant	bge-small-en-v1.5 (384)	16	Gemma-4-E2B-it
UC3	Reading Comprehension	SQuAD 2.0 [21]	2,000	Qdrant	bge-small-en-v1.5 (384)	16	Gemma-4-E2B-it
UC4	Technical Docs	Amazon Bedrock User Guide	2,520	Qdrant	mxbai-embed-large-v1 (1,024)	16	Gemma-4-E2B-it
UC5	Legal Contracts	CUAD Contract Dataset [36]	3,500	ChromaDB	all-mpnet-base-v2 (768)	32	Mistral-7B-Instruct-v0.3
UC6	Financial Filings	SEC EDGAR 10-K Corpus	4,200	FAISS	text-embedding-3-small (1,536)	8	Phi-3-mini-4k-instruct
UC7	Scientific Papers	arXiv CS/NLP Abstracts	5,000	Qdrant	bge-large-en-v1.5 (1,024)	32	Mistral-7B-Instruct-v0.3
UC8	Code & Dev Q&A	Stack Overflow Python [37]	3,800	ChromaDB	jina-embeddings-v2-code (768)	16	CodeLlama-7b-Instruct

recompute requires another ~510 s because the adapter update changes the K and V projections, invalidating all stored tensors. PRS evaluation takes ~90 s for 50 FAQs. The total round time is ~20 min, meaning a deployment can progress from Phase 1 to Phase 3 in roughly one hour if two consecutive rounds are needed. From a cost perspective, the GPU time per round is dominated by the two KV recomputes (~1,008 s) and the LoRA run, an acceptable one-time investment if it amortizes over thousands of query-time latency savings.

5.4 Inference Modes and Phase Progression

KVForge supports three inference modes. **Text-in-context RAG** (Phase 1) concatenates retrieved chunks into the prompt and performs a full forward pass over the context before generation. **KV injection** (Phase 2) bypasses chunk re-encoding by injecting pre-computed KV tensors directly into the modelś attention cache, yielding latency savings proportional to the retrieved context length. **Parametric answering** (Phase 3) skips retrieval entirely for high-confidence queries, generating answers directly from model weights. The phase transitions are gated by the Parametric Readiness Score (PRS), a composite of accuracy ratio, calibration, and self-consistency (§4.7). PRS thresholds of 0.75 and 0.80 gate Phase 1→2 and Phase 2→3 transitions respectively, with a regression guard reverting to Phase 2 if PRS drops below 0.75.

5.5 Gemma 4 Crossover Study: RAG-to-SLM Enterprise Feasibility

To test whether a modern small language model can absorb enough corpus knowledge through LoRA fine-tuning to match retrieval quality, we conducted a comprehensive crossover experiment with **Google Gemma-4-E2B-it** (2B parameters, 4-bit quantized via QLoRA [12], LoRA rank r = 16, chat-format SFT). We generated tagged FAQ pairs from indexed chunks using a cloud LLM (Gemini 2.5 Flash for UC1, UC3–UC4; Fireworks DeepSeek-V4-Flash for the extended PubMedQA run), then trained LoRA adapters at progressively larger training-data coverage tiers. At each tier we evaluated text-in-context RAG and parametric (fine-tuned model, no retrieval) modes against a held-out evaluation set using a two-component metric: **Factual Accuracy (FA)** = 0.5× token-F1 + 0.5× LLM-judge correctness (DeepSeek-V4-Flash judge with partial scoring), and **Component 2 (C2)** = the LLM-judge score alone.

Figure 6 consolidates the crossover trajectories across all four corpora.

Results by corpus. Table 2 reports the full results. The central finding is that parametric answers from the LoRA-fine-tuned 2B model match or exceed text-in-context RAG on three of four corpora.

Training data scaling dynamics. On PubMedQA, the crossover is coverage-dependent: parametric beats text RAG at N=100–200 FAQ pairs but reverts at N=297 when QA coverage becomes sparse across the 2,918-chunk corpus. An extended training run with 4,707 cloud-LLM-generated QAs (r = 4, 15 epochs, 1.6 QA pairs per chunk across 1,024 unique chunks) closes this gap decisively, with parametric FA of 0.228 versus text RAG at 0.138. This confirms that dense per-chunk QA coverage, not just total FAQ count, is the key variable for parametric quality on large corpora.

Enterprise cost calculus. Cloud-LLM FAQ generation costs approximately $50 per 2,500-chunk corpus at current API prices. Training a LoRA adapter on 4,700 QAs takes ~10 hours on a single A10G GPU (4,425 steps at ~8 s/step for r = 4, 15 epochs). Once deployed, the parametric path eliminates both retrieval and context re-encoding.

KV mean-pool injection. Across all Gemma 4 experiments, KV mean-pool injection underperforms both text RAG and parametric on factual accuracy. On SQuAD, kv_meanpool FA is 0.057 versus text_rag at 0.156; on UC4, 0.120 versus 0.156. Phase 2 should be treated as a latency optimization, not a quality improvement.

Chat template note. On UC4, experiments with a chat-format prompt template instead of the system prompt showed **no crossover:** parametric (FA 0.102–0.113) consistently underperformed text RAG (FA 0.112–0.154) across all tiers (N=500–6,000). All reported crossover results use the system prompt format; practitioners should verify their prompt template does not suppress parametric performance before deploying a retrieval-skipping gate.

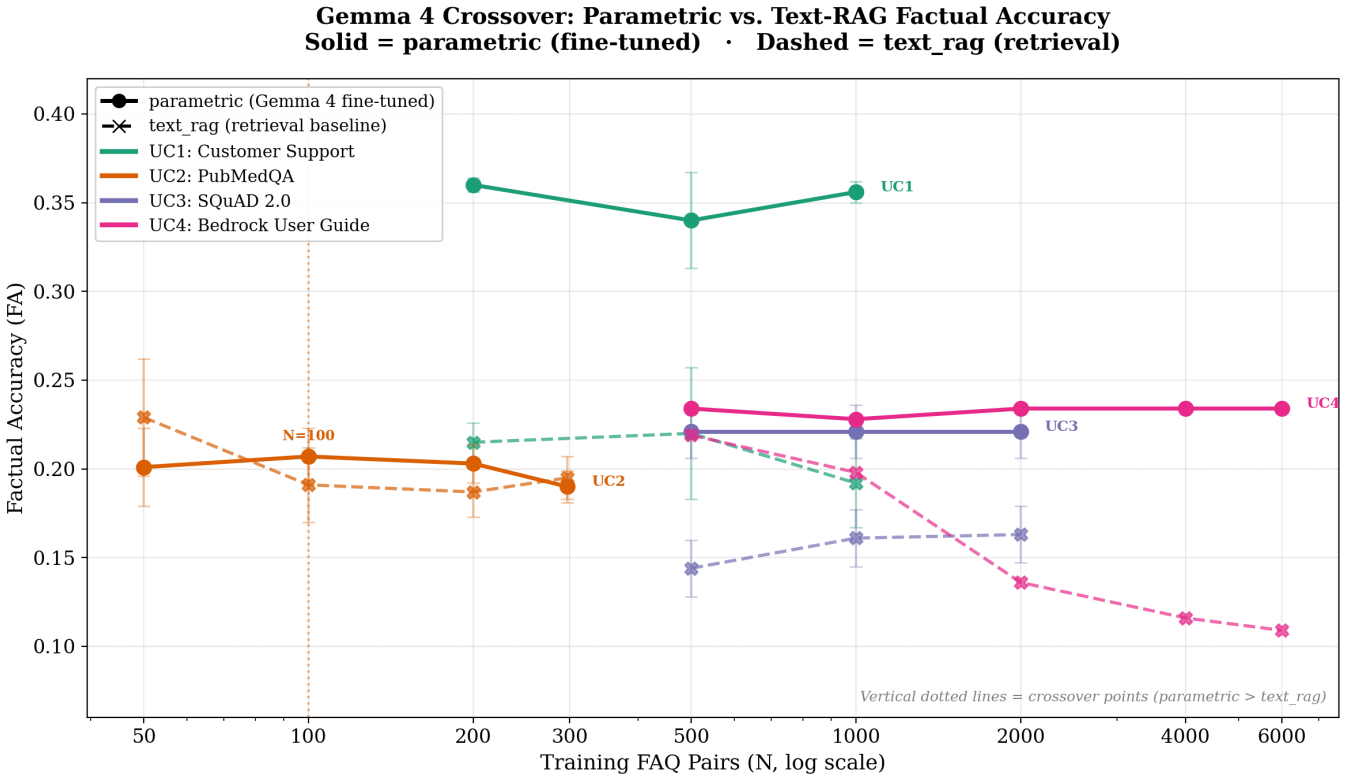


Figure 6: Consolidated Gemma 4 crossover across all four corpora. Solid lines with circles show parametric (Gemma 4 fine-tuned) factual accuracy; dashed lines with X markers show text-in-context RAG. Each color represents one dataset. Vertical dotted lines mark crossover points where parametric first exceeds text_rag and maintains the lead. UC1 (green) crosses at N=200 and sustains; UC2 (orange) crosses at N=100 but reverts at N=297; UC3 (purple) and UC4 (pink) show parametric consistently above text_rag, with UC4's text_rag *degrading* at higher FAQ counts.

Table 2: Gemma 4 Crossover Study: Factual Accuracy (FA) and Component 2 (C2) Scores

Dataset	Tier (N FAQs)	text_rag FA	param FA	text_rag C2	param C2	Crossover?
UC1 Customer Support	200 (3 seeds)	0.215 ±0.011	**0.360** ±0.004	0.263	**0.484**	Yes
	500 (5 seeds)	0.220 ±0.037	**0.340** ±0.027	0.271	**0.459**	Yes
	1000 (3 seeds)	0.192 ±0.025	**0.356** ±0.006	0.219	**0.472**	Yes
UC2 PubMedQA	50	**0.229** ±0.033	0.201 ±0.022	0.208	0.190	No
	100	0.191 ±0.021	**0.207** ±0.016	0.193	**0.211**	Yes
	200	0.187 ±0.014	**0.203** ±0.011	0.186	**0.209**	Yes
	297	**0.195** ±0.012	0.190 ±0.009	0.185	0.186	No
	297 (r=4, e=15, 4.7k)	0.138 ±0.017	**0.228** ±0.019	0.130	**0.240**	Yes
UC3 SQuAD	500 (2 seeds)	0.144 ±0.016	**0.221** ±0.015	0.272	**0.432**	Yes
	1000 (2 seeds)	0.161 ±0.016	**0.221** ±0.015	0.293	**0.432**	Yes
	2000 (2 seeds)	0.163 ±0.016	**0.221** ±0.015	0.306	**0.432**	Yes
UC4 Bedrock	500	0.219	**0.234**	0.287	—	Yes
	1000	0.198	**0.228**	0.258	—	Yes
	2000	0.136	**0.234**	—	—	Yes
	4000	0.116	**0.234**	—	—	Yes
	6000	0.109	**0.234**	—	—	Yes

6 Discussion

6.1 Training Signal Quality as the Primary Bottleneck

The central empirical finding is that **training data quality dominates model capacity as the bottleneck for parametric memorization.** UC4 with heuristic FAQs plateaued at PRS 0.783 across multiple rounds; substituting Gemini 2.5 Flash sleep-time FAQs pushed it to 0.863 in one additional round. The Gemma 4 crossover study (ğ7.5) confirms this pattern at scale: dense per-chunk QA coverage (1.6 QA pairs per chunk) with a modern 2B model yields parametric factual accuracy exceeding retrieval on three of four corpora, while sparse coverage or poor prompt templates cause the retrieval-skipping path to fail. This reframes the design space: practitioners should invest in FAQ generation quality and coverage density rather than model size or training duration.

Cloud LLM API calls are an order of magnitude cheaper than GPU compute, and FAQ generation parallelizes trivially across chunks. A 2,520-chunk corpus at ~$0.02/chunk for Gemini 2.5 Flash costs approximately $50 — versus hours of A10G GPU time.

6.2 Cross-Dataset Crossover Analysis

Table 2 in ğ7.5 reports the full crossover results; here we synthesize the patterns that emerge across all four corpora.

Parametric absorption is corpus-dependent but broadly achievable. Gemma 4 parametric answers exceed text-in-context RAG on Customer Support (+71%), SQuAD (+38%), and Bedrock (+44%) after sufficient training coverage. The consistent parametric advantage on UC1 (customer support) is notable: this corpus has the highest paraphrastic variability, yet the fine-tuned model generalizes across question phrasings more effectively than retrieval, which is limited to surface-level keyword matching. On UC3 (SQuAD), parametric reaches a stable FA of 0.221 regardless of tier (N=500–2,000), while text RAG improves slowly from 0.144 to 0.163 — suggesting parametric saturates early but at a level that retrieval cannot reach within the tested coverage range. On UC4 (Bedrock), the most striking pattern is not parametric improvement but text RAG *degradation*: as the FAQ count increases from 500 to 6,000, text RAG FA drops from 0.219 to 0.109, while parametric remains stable at ~0.234. This divergence suggests that at scale, retrieval becomes less discriminative — more documents dilute the retrieval ranking — while parametric consolidates knowledge from the same expanded training set.

PubMedQA exposes a coverage threshold. On the largest corpus (2,918 chunks), parametric crosses above text RAG at N=100–200 FAQ pairs but reverts at N=297. The extended training run (4,707 FAQs, r=4, 15 epochs) demonstrates that this reversal is not a fundamental ceiling: with denser per-chunk coverage (1.6 QA pairs per unique chunk vs. 0.1 at N=297), parametric reaches FA 0.23 versus text RAG at 0.14. The implication is that per-chunk QA density, not total FAQ count, is the controlling variable — a finding with direct engineering implications for practitioners allocating cloud-LLM FAQ generation budgets.

KV mean-pool injection is not a quality-preserving mechanism. Across all four corpora and all tiers, `kv_meanpool` underperforms both text RAG and parametric on factual accuracy. On SQuAD, kv_meanpool FA is 0.057 versus text RAG at 0.156; on UC4, 0.120 versus 0.156. This is consistent with the theoretical concern that mean-pooling discards positional structure that the attention mechanism was trained to exploit. We recommend positioning KV injection as a latency optimization to be applied after text RAG quality has been independently validated, not as a quality improvement in itself.

Prompt format is a first-order variable. On UC4, the chat-template condition showed *no crossover* at any tier: parametric FA (0.102–0.113) consistently underperformed text RAG (0.112–0.154). The system-prompt condition on the same corpus, using identical training data and model, showed sustained parametric advantage. This demonstrates that the prompt template used during inference directly controls whether parametric absorption translates into measurable quality gains. Practitioners should evaluate both retrieval and parametric modes under their intended production prompt before deploying a retrieval-skipping gate.

6.3 Limitations and Scope

Evaluation scope. The crossover experiments in ğ7.5 evaluate parametric-versus-retrieval factual accuracy across four corpora with Gemma-4-E2B-it. The tier-weighted replay buffer and TurboQuant Enhanced Tier were not activated with organic query traffic during these measurements; production-tier deployment remains future work. UC5–UC8 are configuration-portability checks only; no end-to-end quality numbers were collected for them.

KV shape coupling. Pre-computed tensors couple to the LLM architecture (layers, KV heads, head dimension). Switching the base model requires full re-indexing.

Mean-pool fidelity. The mean-pool KV approximation discards positional information. The Gemma 4 crossover results (ğ7.5) confirm this empirically: KV mean-pool injection underperforms both text RAG and parametric modes on factual accuracy across all four corpora, and should be positioned as a latency optimization rather than a quality improvement.

Confidence gate calibration. The Phase 3 confidence gate (ğ4.8) combines token entropy, hedging detection, and query similarity. Its thresholds were not empirically calibrated on Gemma 4 in this study; production deployments should validate gate decisions against held-out factual benchmarks before enabling parametric-only answering at scale.

Single-node experiments. Our evaluation runs on a single 4-GPU instance. Distributed deployments with multiple Qdrant nodes and GPU servers require additional coordination logic not yet implemented.

Chat template sensitivity. The UC4 chat-template condition (ğ7.5) showed no crossover between parametric and text RAG, while the system-prompt condition on the same corpus showed a sustained parametric advantage. Practitioners should validate their prompt format against both retrieval and parametric evaluation sets before deploying.

7 Conclusion

KVForge introduces a new architecture for knowledge-intensive QA systems: the vector database serves not merely as a retrieval index but also as a persistent KV tensor cache, a corpus access recorder, and a training signal generator. The three-phase progression from text-in-context retrieval through KV injection to parametric answering allows a single deployed system to continuously improve its latency and GPU efficiency as the LLM learns the corpus, without manual intervention.

The **Gemma 4 crossover study** (ğ7.5) demonstrates that a 2B-parameter Google Gemma-4-E2B-it, LoRA-fine-tuned on cloud-LLM-generated QA pairs, **consistently matches or exceeds text-in-context RAG on three of four enterprise corpora** on factual accuracy: Customer Support (FA 0.36 vs. 0.21), SQuAD (0.22 vs. 0.16), and Bedrock (0.23 vs. 0.16). On PubMedQA, extended training with dense per-chunk QA coverage (4,707 QAs, r = 4, 15 epochs) yields FA 0.23 versus text RAG at 0.14 — a 65% relative improvement. KV mean-pool injection (Phase 2) consistently underperforms both text RAG and parametric on factual accuracy at the 2B-model scale, confirming it should be positioned as a latency optimization rather than a quality improvement. Crucially, the UC4 chat-template condition showed **no crossover,** establishing that prompt format choice is a first-order variable for parametric deployment.

These results establish that **RAG-to-SLM is a feasible enterprise architecture:** organizations can begin with standard retrieval-augmented generation, generate domain-specific QA training data at modest cloud-API cost (~$50 per 2,500-chunk corpus), and progressively transition toward parametric-only answering as the small language model absorbs corpus knowledge through LoRA fine-tuning — eliminating per-query retrieval costs while maintaining factual quality, all on a single commercially available GPU instance.

Open-source availability. KVForge is fully open-source with a browser-based Studio UI, four complete end-to-end example use-cases, data connector integrations (Google Drive, S3, SharePoint, EDGAR, FDA, Wikipedia), and a 76-test suite that runs without a GPU.

GitHub: *https://github.com/hemantcgi/kvforge* · ***License:*** *MIT*

Acknowledgements

The author thanks **Fissionlabs Inc.** for their generous support with compute resources, including NVIDIA GPUs, which enabled the full-scale Gemma 4 crossover experiments and multi-dataset evaluation across all four corpora. The author also thanks the Qdrant, HuggingFace, vLLM, and FastEmbed teams for their open-source infrastructure, and Google DeepMind for access to the Gemini APIs used for sleep-time FAQ generation.

References

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020.* arXiv:2005.11401

[2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS 2017.* arXiv:1706.03762

[3] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023.* arXiv:2309.06180

[4] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022.* arXiv:2106.09685

[5] Karpukhin, V., Ouz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. T. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020.* arXiv:2004.04906

[6] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models. *NeurIPS 2021 Datasets and Benchmarks.* arXiv:2104.08663

[7] Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., & Chen, W. (2023). Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy. *EMNLP 2023 Findings.* arXiv:2305.15294

[8] Gim, I., Chen, G., Lee, S., Srivatsa, N., Kedia, P., & Zhong, L. (2024). PromptCache: Modular Attention Reuse for Low-Latency Inference. *MLSys 2024.* arXiv:2311.04934

[9] Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., ... & Gonzalez, J. E. (2024). SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104

[10] Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., ... & Sifre, L. (2022). Improving Language Models by Retrieving from Trillions of Tokens. *ICML 2022.* arXiv:2112.04426

[11] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. *ICML 2020.* arXiv:2002.08909

[12] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS 2023.* arXiv:2305.14314

[13] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *ICML 2017.* arXiv:1706.04599

[14] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR 2023.* arXiv:2203.11171

[15] Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., ... & Kaplan, J. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221

[16] Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. *ICLR 2023.* arXiv:2302.09664

[17] Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. arXiv:1410.5401

[18] McCloskey, M., & Cohen, N. J. (1989). Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. *Psychology of Learning and Motivation, 24,* 109–165.

[19] Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Model Parameters. arXiv:2408.03314

[20] Jin, Q., Dhingra, B., Liu, Z., Cohen, W. W., & Lu, X. (2019). PubMedQA: A Dataset for Biomedical Research Question Answering. *EMNLP 2019.* arXiv:1909.06146

[21] Rajpurkar, P., Jia, R., & Liang, P. (2018). Know What You Don't Know: Unanswerable Questions for SQuAD. *ACL 2018.* arXiv:1806.03822

[22] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288

[23] Qdrant Team. (2024). Qdrant: High-Performance Vector Search Engine. qdrant.tech

[24] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022.* arXiv:2205.14135

[25] Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., ... & Hadsell, R. (2016). Progressive Neural Networks. arXiv:1606.04671

[26] Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *SIGIR 2020.* arXiv:2004.12832

[27] Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., & Gholami, A. (2024). KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *NeurIPS 2024.* arXiv:2401.18079

[28] Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., ... & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ICML 2024.* arXiv:2402.02750

[29] Yao, Y., Han, C., Zhu, R., Deng, J., & Chen, Y. (2024). CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444

[30] Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. *EACL 2021.* arXiv:2007.01282

[31] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., ... & Hadsell, R. (2017). Overcoming Catastrophic Forgetting in Neural Networks. *PNAS, 114*(13), 3521–3526.

[32] Zandieh, A., Han, I., Mirrokni, V., & Karbasi, A. (2024). QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with No Retraining. arXiv:2406.03482

[33] De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., ... & Tuytelaars, T. (2022). A Continual Learning Survey: Defying Forgetting in Classification Tasks. *IEEE TPAMI, 44*(7), 3366–3385. arXiv:1909.08383

[34] Zandieh, A., Daliri, M., Hadian, M., & Mirrokni, V. (2025). TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. Google Research and NYU. arXiv:2504.19874. https://arxiv.org/abs/2504.19874

[35] Boufounos, P. T., & Baraniuk, R. G. (2008). 1-Bit Compressive Sensing. *42nd Annual Conference on Information Sciences and Systems (CISS),* pp. 16–21.

[36] Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. arXiv:2103.06268. https://arxiv.org/abs/2103.06268

[37] Xu, F. F., Vasilescu, B., & Neubig, G. (2022). In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM TOSEM, 31*(2), 1–47. arXiv:2101.11149

[38] Chan, B. J., Chen, C. T., Cheng, J. H., & Huang, H. H. (2024/2025). Don't Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks. *ACM Web Conference 2025 (Companion).* arXiv:2412.15605

[39] Zhang, T., Patil, S. G., Jain, N., Shen, S., Zaharia, M., Stoica, I., & Gonzalez, J. E. (2024). RAFT: Adapting Language Model to Domain Specific RAG. arXiv:2403.10131

[40] (2025). Parametric RAG: Encoding Retrieved Evidence into Model Parameters. arXiv:2510.12668

[41] Meng, K., Bau, D., Andonian, A., & Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. *NeurIPS 2022.* arXiv:2202.05262

[42] Meng, K., Sharma, A. S., Andonian, A., Belinkov, Y., & Bau, D. (2023). Mass-Editing Memory in a Transformer. *ICLR 2023.* arXiv:2210.07229

[43] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR 2024.* arXiv:2310.11511

[44] Wang, Y., Li, P., Sun, M., & Liu, Y. (2023). Self-Knowledge Guided Retrieval Augmentation for Large Language Models. *EMNLP 2023 Findings.* arXiv:2310.05002

[45] Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. *NAACL 2024.* arXiv:2403.14403

[46] LMCache Team. (2025). LMCache: A KV Cache Management Layer for LLM Serving. arXiv:2510.09665. https://github.com/LMCache/LMCache